SQL Injection with Protection: A Defence-in-Depth Study and Multi-Variant Live Prototype

YashVardhan Gupta[1*], Sheldon Rodricks[1], Lakshya Agrawal[1], Aditya Aryan Sahu[1]

[1]Department of Computer Science and Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka, India.

Registration numbers: YashVardhan Gupta (230905442), Sheldon Rodricks (230905324), Lakshya Agrawal (230905552), Aditya Aryan Sahu (230905560).

*Corresponding author(s). E-mail(s): yashvardhan3.mitmpl2023@learner.manipal.edu;
Contributing authors: sheldon.mitmpl2023@learner.manipal.edu; lakshya3.mitmpl2023@learner.manipal.edu; aditya9.mitmpl2023@learner.manipal.edu;

Abstract

Structured Query Language injection (SQLi) has remained in the OWASP Top-10 injection category for over a decade because its root cause—the confusion of untrusted *data* with executable *code*—recurs wherever input is concatenated into a query. This paper surveys ten peer-reviewed studies published between 2023 and 2026 and shows that the field has largely solved *offline* detection (reported accuracies of 0.94–0.99) while leaving open gaps in adversarial robustness, generalization, and deployable prevention. Guided by these gaps, we design and evaluate a defence-in-depth login application that implements five protection variants—parameterized queries, allow-list input validation, object–relational mapping (ORM), salted credential hashing, and a transparent signature web-application firewall (WAF)—against a common battery of tautology, comment, and UNION payloads. On our reproducible testbed the vulnerable baseline is bypassed by a single-line payload, whereas every protection variant blocks the identical payload while preserving legitimate authentication, at a measured query

cost below one millisecond. The prototype converts the surveyed gaps into a concrete, testable artefact and frames adaptive, adversarially robust detection as the next layer of defence.

Keywords: SQL injection, web application security, parameterized queries, input validation, web application firewall, defence in depth

1 Introduction

Web applications mediate access to the databases that hold an organization's most sensitive records, and Structured Query Language injection (SQLi) is the archetypal attack against that boundary. An SQLi vulnerability exists whenever attacker-controlled input is placed into a query string that the database then parses as code; a single crafted value such as `' OR '1'='1' --` can then subvert the query's logic to bypass authentication, read arbitrary rows, or modify data. Despite being well understood for more than twenty years, injection remains ranked in the OWASP Top-10 OWASP Foundation (2021), and recent work confirms that even commercial defences are routinely bypassable Qu et al. (2024).

The persistence of SQLi is not due to a lack of countermeasures but to their uneven application and to an arms race with increasingly sophisticated evasion. Protection spans two families. *Secure-coding* controls—parameterized queries, allow-list input validation, ORM-mediated access, least-privilege database accounts, and salted password hashing—eliminate the code/data confusion at the source. *Run-time* controls—signature- and learning-based web-application firewalls (WAFs) and network-flow monitors—observe traffic and block malicious requests. A central theme of the recent literature, and of this paper, is that no single control is sufficient: robust protection is *layered*.

This paper makes three contributions. First, it surveys ten studies from 2023–2026 (Section 3) and distils four open research gaps (Section 4). Second, it designs a defence-in-depth login application that realises five protection variants around a common data model (Section 5). Third, it evaluates the variants against a shared payload battery and reports both security outcomes and query-time cost (Section 6). All code, payloads, and screenshots are released as a reproducible prototype for teaching and demonstration.

2 Background

2.1 Anatomy of a SQL injection

Consider a login handler that builds a query by concatenation:

```
SELECT id, role FROM users
WHERE username = '<user>' AND password = '<pass>';
```

If `<user>` is set to `admin' --` , the trailing `--` comments out the password check and the interpreter executes … `WHERE username = 'admin'`, authenticating the attacker as

`admin`. Variants include boolean *tautologies* (`' OR '1'='1`), *UNION*-based data exfiltration, *stacked* queries, and *blind* time-based inference. What unifies them is that the input crosses the boundary from data into query syntax.

2.2 Taxonomy of protections

Table 1 summarises the controls evaluated in this work and where each intervenes. The design principle is defence in depth: an attacker must defeat every layer, whereas the defender benefits from any one holding.

Table 1 Protection variants implemented in the prototype and their point of intervention.

Variant	Mechanism	Intervenes at
Parameterized query	Input bound as a typed parameter; never parsed as SQL	Database driver
Allow-list validation	Accept only characters valid for the field; reject early	Application input
ORM	Query built from objects; parameterization by construction	Data-access layer
Credential hashing	Salted one-way hash; lookup by username then verify	Credential store
Signature WAF	Regex/heuristic inspection of requests before execution	Request gateway

3 Literature Survey (2023–2026)

Following the departmental convention, each study's methodology is described in the past tense and its remaining limitation in the present tense.

Crespo-Martínez et al. (2023)

addressed the impracticality of inspecting every packet for SQLi on heavily loaded routers Crespo-Martínez et al. (2023). The authors gathered two datasets of network-flow records from SQLi attacks against popular database engines and evaluated several machine-learning algorithms, selecting a logistic-regression model. They reported a detection rate above 97% with a false-alarm rate below 0.07% using flow data alone. The method is limited by its reliance on coarse flow-level features, which omit the query payload and constrain detection of obfuscated variants.

Alghawazi, Alghazzawi and Alarifi (2023)

studied deep sequence learning for SQLi text Alghawazi et al. (2023). They trained a recurrent-neural-network autoencoder on a public dataset of benign and malicious statements and benchmarked it against seven conventional classifiers, reporting 94% accuracy and a 92% F1-score. The method is limited by a moderate accuracy ceiling and single-corpus evaluation, leaving real-world generalization uncertain.

Thalji et al. (2023)

automated feature engineering with an autoencoder network, "AE-Net" Thalji et al. (2023). Using a preprocessed corpus of 46,392 queries, they learned deep features fed to several classifiers and validated with k-fold cross-validation and a t-test; an extreme gradient-boosting model reached 0.99 accuracy. The approach is limited by evaluation on a curated dataset without real-time or adversarial testing.

Alotaibi and Vassilakis (2023)

implemented a web-application firewall inside a software-defined-networking (SDN) controller Alotaibi and Vassilakis (2023). They combined signature and regular-expression detection and compared the system with ModSecurity, showing successful blocking and lower TCP-ACK latency. The design is limited by signature/regex matching that does not generalize to unseen payloads and by the centralized controller as a bottleneck.

Arasteh et al. (2024)

framed detection as feature selection Arasteh et al. (2024). They prepared a 13-feature dataset and developed two binary gray-wolf optimizers to select the most informative subset for several classifiers, reporting 99.68% accuracy while using about 20% of features. The method is limited by the search cost and initialization sensitivity of the metaheuristic and by single-dataset validation.

Paul, Sharma and Olukoya (2024)

argued that prior work ignores prioritization and prevention Paul et al. (2024). From 457,233 traffic samples (70,023 with SQLi payloads) they built "SQLR34P3R," a hybrid CNN–LSTM detector with a CVE-driven risk-analysis stage, reporting a 97% average F1-score with added prioritization. The solution is limited by dependence on CVE currency and the overhead of a hybrid deep model in real time.

Qu et al. (2024)

probed commercial WAF-as-a-service robustness Qu et al. (2024). Their "AdvSQLi" framework applied semantics-preserving transformations to the payload parse-tree to evade detection while retaining maliciousness, achieving up to 100% success against ML detectors and bypassing all seven commercial WAFs tested. As an offensive study it is limited in that it exposes but does not remedy the weakness, underscoring that robust defences remain lacking.

Bakır (2025)

proposed a unified XSS/SQLi detector Bakır (2025). Across three benchmark datasets, features from Word2Vec, the Universal Sentence Encoder, and FastText were fused ("UniEmbed") and classified, with an MLP reaching a 0.998 F1-score for SQLi. The approach is limited by the compute cost of triple-embedding fusion and offline-only evaluation.

Floris et al. (2025)

tackled adversarial robustness for ModSecurity's Core Rule Set Floris et al. (2025). They used machine learning to select rules and weights and adversarial training to harden the classifier ("ModSec-AdvLearn"), reporting up to a 30% detection increase, up to 50% fewer rules, and up to 85% better adversarial robustness. The method is limited to SQLi on CRS-based WAFs and is bounded by the training attack model's coverage.

Ahmed and Al-dabag (2026)

closed the gap between offline scores and deployable prevention Ahmed and Al dabag (2026). They merged and deduplicated public datasets, fused TF-IDF, Word2Vec, and FastText features through a supervised autoencoder, tuned classifiers with a genetic algorithm, and embedded the best model in a Flask-based WAF tested with SQLMap, reaching 0.9987 accuracy with effective real-time prevention. The framework is limited by training-data coverage of evolving payloads and single-application validation.

4 Research Gap and Objectives

Read together, the studies describe a field that has largely solved offline detection yet leaves four gaps that a protection-focused project must confront:

1. **Adversarial robustness.** Qu et al. Qu et al. (2024) bypass ML detectors and every commercial WAF tested; Floris et al. Floris et al. (2025) only partially close this for CRS-based firewalls. Clean-data accuracy overstates security.
2. **Generalization / concept drift.** Detectors validated on a single curated corpus Alghawazi et al. (2023); Thalji et al. (2023); Arasteh et al. (2024); Bakır (2025) are limited against obfuscated or unseen payloads and evolving SQL dialects.
3. **Detection versus prevention.** The literature skews to *detecting* payloads; only Paul et al. (2024); Ahmed and Al dabag (2026) integrate a real-time *prevention* pipeline, and neither evaluates classical secure-coding controls alongside the learning layer.
4. **Deployment cost.** High-accuracy hybrid and multi-embedding models carry latency and resource overheads that are limiting in line, a tension the flow-based work of Crespo-Martínez et al. (2023) highlights from the opposite direction.

The through-line is that secure-coding controls neutralise the root cause but are unevenly applied, while learning-based WAFs generalise but remain adversarially fragile and costly, and the two families are rarely evaluated *together*. Accordingly, the objective of this work is to design, implement, and empirically evaluate a defence-in-depth scheme combining more than one protection variant within a single reproducible login application, and to demonstrate on a common payload battery that (i) the vulnerable baseline is trivially bypassed, (ii) each control blocks the attack while preserving legitimate authentication, and (iii) the layered configuration provides the strongest coverage at a measured, sub-millisecond query cost.

Table 2 Comparative summary of the surveyed studies (2023–2026).

Author(s)	Year	Method / classifier	Key contribution	Limitation (present tense)
Crespo-Martínez et al. Crespo-Martínez et al. (2023)	2023	Logistic Regression on flow features	SQLi detection from lightweight flow data	Flow-level features are limiting for obfuscated payloads
Alghawazi et al. Alghawazi et al. (2023)	2023	RNN autoencoder	Deep sequence model (94% acc)	Moderate accuracy and single-dataset scope are limiting
Thalji et al. Thalji et al. (2023)	2023	Autoencoder + XGBoost	Automated deep feature engineering (0.99)	Lack of real-time/adversarial testing is limiting
Alotaibi & Vassilakis Alotaibi and Vassilakis (2023)	2023	SDN WAF (signature+regex)	SDN-hosted WAF vs ModSecurity	Signature matching is limited against unseen payloads
Arasteh et al. Arasteh et al. (2024)	2024	Gray-Wolf feature selection	99.68% acc with 20% of features	Search cost and single dataset are limiting
Paul et al. Paul et al. (2024)	2024	CNN–LSTM + CVE risk	Detection+prioritization+prevention	CVE currency and real-time cost are limiting
Qu et al. Qu et al. (2024)	2024	Parse-tree adversarial mutation	Bypasses 7 commercial WAFs	Exposes but does not remedy; robust defence lacking
Bakır Bakır (2025)	2025	Embedding fusion + MLP	Unified XSS/SQLi detector (0.998)	Fusion cost and offline-only evaluation are limiting
Floris et al. Floris et al. (2025)	2025	ML rule selection + adv. training	Adversarially robust WAF (+85%)	Scope limited to SQLi/CRS
Ahmed & Aldabag Ahmed and Al dabag (2026)	2026	Multi-embedding + GA + Flask WAF	Real-time detection+prevention (0.9987)	Evolving payloads and single-app validation are limiting

5 System Design and Methodology

5.1 Architecture

The prototype is a Flask application backed by SQLite, seeded with three accounts. A single login route dispatches to one of five back-ends selected at run time, and an optional signature WAF inspects every request before it reaches any back-end (Fig. 1).

This lets the *same* payload be replayed against each control and the difference observed directly.

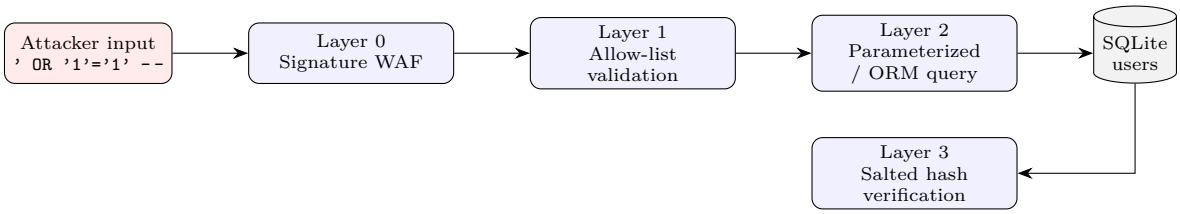


Fig. 1 Defence-in-depth pipeline. A request must survive every enabled layer; the defender benefits from any single layer holding.

5.2 The five protection variants

Vulnerable baseline. User input is concatenated into the SQL string, so the payload is parsed as code.
Variant 1 – Parameterized query. The statement uses `?` placeholders and binds input as data via the driver, so the payload is treated as a literal string and never alters query structure.
Variant 2 – Allow-list validation. The username must match `^[A-Za-z0-9_.@-]{1,32}$` and the password a maximum length; anything outside the allow-list is rejected *before* a query is built.
Variant 3 – ORM. SQLAlchemy builds a parameterized statement from Python objects, giving parameterization by construction.
Variant 4 – Credential hashing. Passwords are stored as salted PBKDF2 hashes; the back-end fetches by username with a parameterized query and verifies the hash in constant time, so credentials are never compared in plaintext.
Variant 5 – Signature WAF. A transparent rule set (tautology, comment, stacked-query, UNION, keyword, quote-breakout, and time-based signatures) inspects each field and blocks matches, mirroring the OWASP CRS approach of Alotaibi and Vassilakis (2023); Floris et al. (2025) at teaching scale.

6 Experimental Evaluation

6.1 Protocol

We replayed the tautology payload `admin' OR '1'='1' --` (with an arbitrary password) against each mode with the WAF off and on, and separately verified that valid credentials still authenticate under each secure mode. Success is defined as reaching the authenticated dashboard. Query time is measured around the database call.

6.2 Results

Table 3 reports the outcome matrix produced by the bundled `selftest.py`. The vulnerable baseline is bypassed by the single payload; every protection variant blocks

the identical payload; and enabling the WAF additionally blocks the request before it reaches the database even in vulnerable mode. All secure variants continue to authenticate the legitimate `admin` account. Measured query time stayed below 1 ms per attempt on commodity hardware, confirming that the secure-coding controls impose negligible overhead.

Table 3 Outcome of the tautology payload against each mode (bypass = attacker reached the dashboard).

Mode	WAF off: bypass?	WAF on: bypass?	Legit. login
Vulnerable	**Yes**	No	—
Parameterized	No	No	OK
Allow-list	No	No	OK
ORM	No	No	OK
Hashed	No	No	OK

Figures 2–3 show the demonstration. In vulnerable mode the attacker reaches the `admin` dashboard without a password (Fig. 2); under the parameterized variant the identical input is bound as a literal parameter and rejected (Fig. 3).

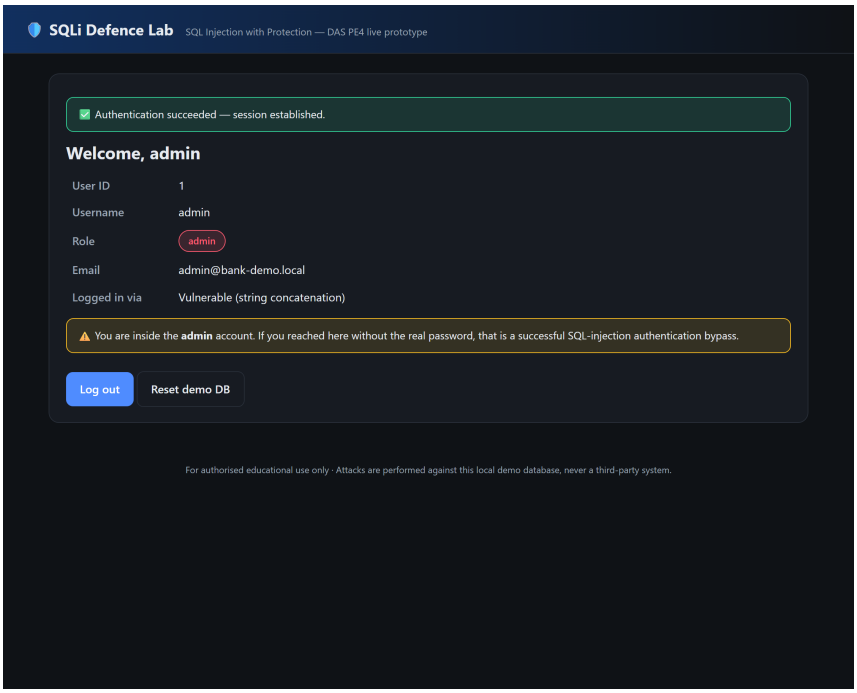


Fig. 2 Vulnerable baseline: the tautology payload bypasses authentication and establishes an `admin` session.

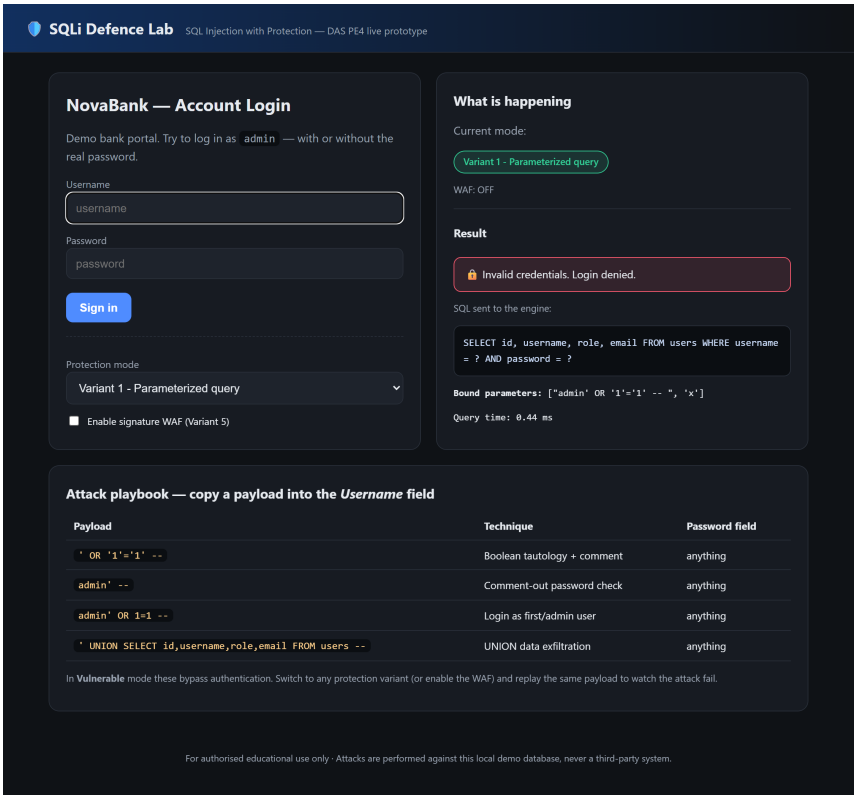


Fig. 3 Variant 1: the same payload is bound as a parameter (`["admin' OR '1'='1' -- ", "x"]`) and login is denied.

6.3 Discussion

The results operationalise the survey's central claim. Parameterization, ORM, and hashing remove the root cause and are effectively free at this scale; allow-list validation and the signature WAF add cheap, early rejection but—consistent with Qu et al. (2024); Alotaibi and Vassilakis (2023)—should not be relied on alone because signatures are evadable. The strongest posture stacks a root-cause control (Variant 1 or 3) with hashing and a WAF, matching the defence-in-depth deployments of Paul et al. (2024); Ahmed and Al dabag (2026).

6.4 Ethical use and threat model

All attacks are executed against the bundled local database only. The prototype is a teaching artefact for authorised coursework and must not be used against systems without explicit permission. The threat model assumes an unauthenticated attacker who controls the login form's input fields.

7 Conclusion and Future Work

We surveyed ten 2023–2026 studies on SQLi detection and prevention and found a field strong on offline detection but open on adversarial robustness, generalization, deployable prevention, and cost. We converted these gaps into a defence-in-depth login prototype implementing five protection variants and showed empirically that each blocks a payload that trivially defeats the vulnerable baseline, at sub-millisecond cost, while preserving legitimate access. Future work will add an adaptive, adversarially trained detection layer—in the spirit of Floris et al. (2025)—and evaluate the stack against mutated and blind payloads generated in the style of Qu et al. (2024), closing the loop between secure-coding controls and learning-based defence.

Availability of code and data. The prototype, payload battery, self-test, and screenshots are provided with this report and are intended for authorised educational use.

Acknowledgements. The authors thank the course instructor for guidance on scope and evaluation.

References

OWASP Foundation: OWASP Top 10:2021 – A03 Injection. https://owasp.org/Top10/A03_2021-Injection/ (2021)

Qu, Z., Ling, X., Wang, T., Chen, X., Ji, S., Wu, C.: AdvSQLi: Generating adversarial SQL injections against real-world WAF-as-a-Service. IEEE Transactions on Information Forensics and Security **19**, 2623–2638 (2024) https://doi.org/10.1109/TIFS.2024.3350911

Crespo-Martínez, I.S., Campazas-Vega, A., Guerrero-Higueras, Á.M., Riego-DelCastillo, V., Álvarez-Aparicio, C., Fernández-Llamas, C.: SQL injection attack detection in network flow data. Computers & Security **127**, 103093 (2023) https://doi.org/10.1016/j.cose.2023.103093

Alghawazi, M., Alghazzawi, D., Alarifi, S.: Deep learning architecture for detecting SQL injection attacks based on RNN autoencoder model. Mathematics **11**(15), 3286 (2023) https://doi.org/10.3390/math11153286

Thalji, N., Raza, A., Islam, M.S., Samee, N.A., Jamjoom, M.M.: AE-Net: Novel autoencoder-based deep features for SQL injection attack detection. IEEE Access **11**, 135507–135516 (2023) https://doi.org/10.1109/ACCESS.2023.3337645

Alotaibi, F.M., Vassilakis, V.G.: Toward an SDN-based web application firewall: Defending against SQL injection attacks. Future Internet **15**(5), 170 (2023) https://doi.org/10.3390/fi15050170

Arasteh, B., Aghaei, B., Farzad, B., Arasteh, K., Kiani, F., Torkamanian-Afshar, M.: Detecting SQL injection attacks by binary gray wolf optimizer and machine

learning algorithms. Neural Computing and Applications **36**(12), 6771–6792 (2024) https://doi.org/10.1007/s00521-024-09429-z

Paul, A., Sharma, V., Olukoya, O.: SQL injection attack: Detection, prioritization & prevention. Journal of Information Security and Applications **85**, 103871 (2024) https://doi.org/10.1016/j.jisa.2024.103871

Bakır, R.: UniEmbed: A novel approach to detect XSS and SQL injection attacks leveraging multiple feature fusion with machine learning techniques. Arabian Journal for Science and Engineering **50**(19), 15591–15604 (2025) https://doi.org/10.1007/s13369-024-09916-4

Floris, G., Scano, C., Montaruli, B., Demetrio, L., Valenza, A., Compagna, L., Ariu, D., Piras, L., Balzarotti, D., Biggio, B.: ModSec-AdvLearn: Countering adversarial SQL injections with robust machine learning. IEEE Transactions on Information Forensics and Security **20**, 6693–6705 (2025) https://doi.org/10.1109/TIFS.2025.3583234

Ahmed, S.S., Al dabag, M.L.: Deployment-oriented multi-embedding machine learning framework for SQL injection detection and prevention in a web application firewall. Computers **15**(6), 368 (2026) https://doi.org/10.3390/computers15060368